

Universidad Tecnológica Nacional
Facultad Regional Buenos Aires

Bases de Datos

Trabajo Práctico
2° Cuatrimestre 2026

FRBA – Gestión de Bicicletería

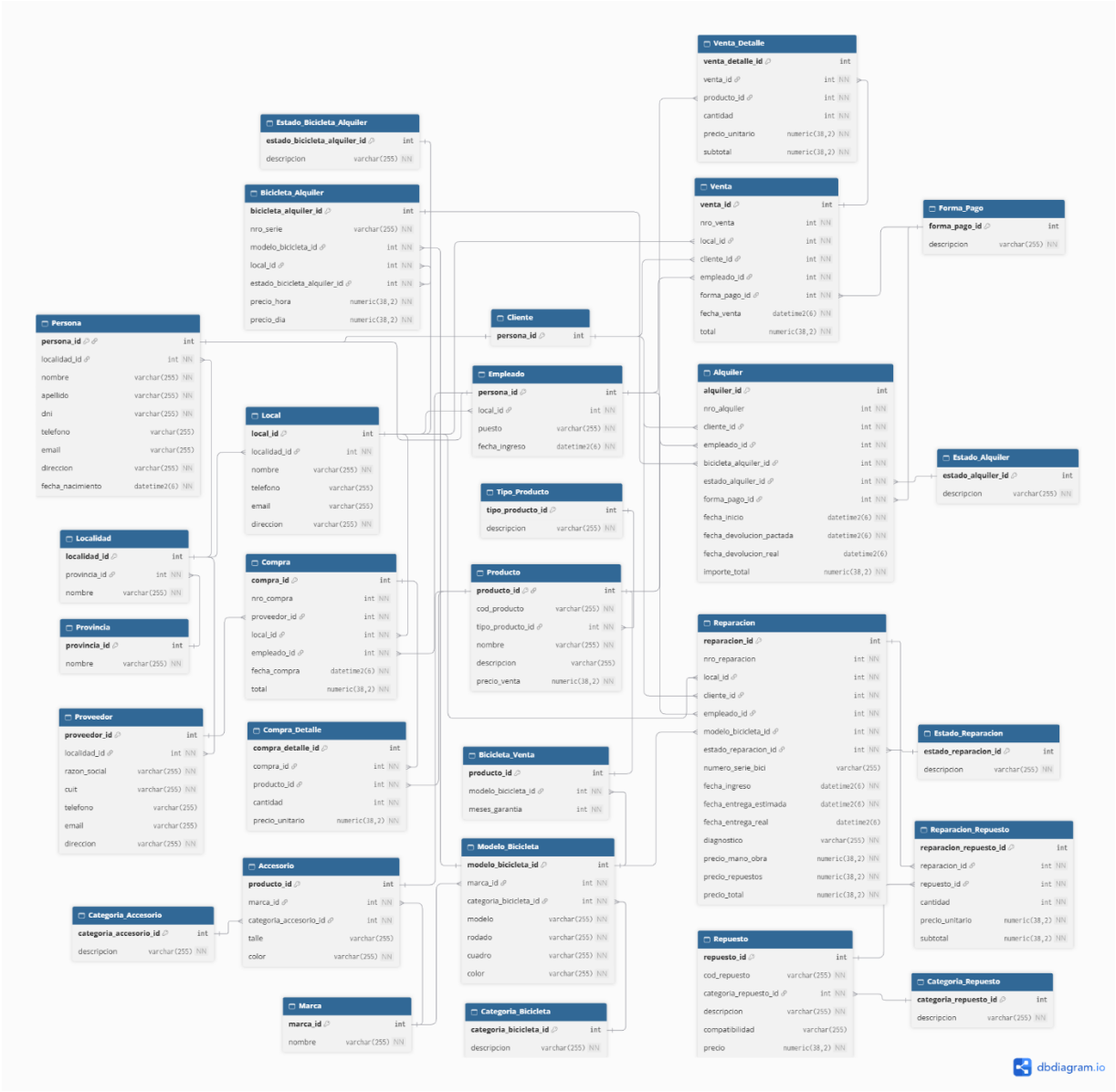
Grupo13 MEGA_DB

Índice

- DER
- Entrega 1
- Entrega 2
    - Criterios de migración
    - Restricciones de integridad
- Inconsistencias
    - Productos repetidos en ventas y repuestos repetidos en reparaciones
    - Error en subtotales de venta
    - Totales de compra
    - DNIs repetidos entre personas distintas
    - Empleado – Local
    - Alquileres superpuestos de la misma bicicleta

Diagrama Entidad Relación

Para mejor definición ver imagen adjunta al .zip



Entrega 1

Consideraciones

- Personas
    - Se decidió modelar una **tabla Persona** para no repetir los datos personales entre Empleado y Cliente.
    - Se decidió modelar una **tabla Cliente**, aunque todos los campos originales de la tabla maestra ya estén en Persona, y **Cliente sea solo una FK a Persona**, a fin de poder distinguir qué personas son clientes usando esta tabla y también poder extenderla a futuro con más campos.
- Localidades
    - Se decidió emplear una **tabla Provincia y otra Localidad**, para tener mejor consistencia de datos respecto de las ubicaciones.
    - Respecto al campo **Dirección** se decidió dejarlo modelado sólo como un **varchar(255),** considerando que no aporta valor profundizar en una entidad para modelarlo y complejizaría innecesariamente el Dominio.
- Producto
    - Se definió una **PK producto_id numérica**, para identificar a los productos por separado del código alfanumérico que ya tenían. Se pretende así optimizar tanto ordenamientos como comparaciones.
    - Se definió que **Producto no incluye Repuestos**, sólo Accesorios y Bicicletas.
- Venta
    - Se definió una **PK venta_id numérica**
- Reparación
    - Se definió una **PK reparacion_id numérica**
    - Se definió la tabla **Reparación_Repuesto** para modelar la relación muchos a muchos entre Reparaciones y Repuestos, y el respectivo desglose de sus detalles.
- Alquiler
    - Se definió una **PK alquiler_id numérica**

Entrega 2

Objetos de base de datos

Todos los objetos se crean dentro del esquema MEGA_DB con el script **script_creacion_inicial.sql** en este orden: tablas, stored procedures, vista, índices y trigger.

- **Tablas**: las 29 tablas del DER con sus respectivos PK, FK y constraints
- **Stored procedures de migración**: 9 SPs donde cada uno carga un grupo de tablas desde gd_esquema.Maestra. Se ejecutan en orden de dependencias dentro de una única transacción.
    - **sp_Migrar_Ubicaciones**: Provincia y Localidad.
    - **sp_Migrar_Locales**: Local.
    - **sp_Migrar_Enums**: Forma_Pago, Marca, Tipo_Producto, categorías y estados.
    - **sp_Migrar_Personas**: Persona, Cliente y Empleado.
    - **sp_Migrar_Productos**: Modelo_Bicicleta, Producto, Bicicleta_Venta, Accesorio y Repuesto.
    - **sp_Migrar_Alquileres**: Bicicleta_Alquiler y Alquiler.
    - **sp_Migrar_Ventas**: Venta y Venta_Detalle.
    - **sp_Migrar_Compras**: Proveedor, Compra y Compra_Detalle.
    - **sp_Migrar_Reparaciones**: Reparacion y Reparacion_Repuesto.
- **Vista**: vw_Estado_Migracion chequea el estado de la migración al finalizar. Para cada tabla nueva se compara su cantidad de filas contra la cantidad de filas esperada de la tabla Maestra. Si las cantidades coinciden indica “OK” sino “REVISAR”.
- **Índices**: son non clustered y se crean después de la migración no ralentizar los Inserts de los sp de migración. Apuntan a las FKs y a las columnas de fecha por las que se filtra y agrupa en las consultas por local, período, producto y bicicleta. Algunos usan INCLUDE para cubrir esas consultas sin ir a la tabla.
    - **Ventas**: IX_Venta_Local_Fecha, IX_Venta_Cliente, IX_Venta_Detalle_Venta, IX_Venta_Detalle_Producto.
    - **Alquileres**: IX_Alquiler_Bicicleta_Fecha, IX_Bicicleta_Alquiler_Local, IX_Bicicleta_Alquiler_Modelo.
    - **Reparaciones**: IX_Reparacion_Local_Ingreso, IX_Reparacion_Repuesto_Reparacion, IX_Reparacion_Repuesto_Repuesto.
    - **Compras**: IX_Compra_Local_Fecha, IX_Compra_Detalle_Compra, IX_Compra_Detalle_Producto.

    - **Catálogo**: IX_Producto_Tipo, IX_Modelo_Bicicleta_Categoria, IX_Repuesto_Categoria.
- **Trigger**: tr_Alquiler_Sin_Superposicion (AFTER INSERT, UPDATE sobre Alquiler) rechaza un alquiler si su rango de fechas se superpone con otro de la misma bicicleta. Un alquiler sin devolución real se considera abierto hasta el infinito (no terminó). Se crea después de la migración para no interferir con los alquileres inconsistentes precargados en Maestra (ver Inconsistencias – Alquileres superpuestos de la misma bicicleta).

Consideraciones

- **Empleado – Local**
    - Se decidió que cada empleado esté asociado al Local donde tiene registradas sus Ventas, Reparaciones y Compras, ignorando donde tiene registrados sus Alquileres. Para más detalle ver Inconsistencias – Empleado – Local.
- **Actualizaciones de tipos de la entrega 1**
    - Se actualizaron algunos tipos de datos del DER de la entrega 1 para no modificar valores durante la migración. Cada columna migrada mantiene el tipo de su columna de origen en gd_esquema.Maestra: DATETIME2(6) para las fechas, NUMERIC(38,2) para los importes, VARCHAR(255) para los textos e INT para los números.
- **Claves subrogadas.** Todas las tablas usan una PK `INT IDENTITY`. Los datos de negocio que identifican a cada entidad (nro de venta, nro de alquiler, código de producto, CUIT, nro de serie, etc.) se protegen con constraints `UNIQUE`.
- **Tablas de dominio.** Estados, categorías, tipos, formas de pago y marcas se normalizan en tablas propias. Los estados de alquiler, de bicicleta de alquiler y de reparación se cargan con `VALUES` usando la lista completa del enunciado, porque la Maestra no trae todos los valores posibles.
- **Supertipos y subtipos.**
    - `Persona` es el supertipo de `Cliente` y `Empleado`.
    - `Producto` es el supertipo de `Bicicleta_Venta` y `Accesorio`.
    - En ambos casos el subtipo usa como PK la misma clave del supertipo (PK = FK).
- **Una sola transacción.** La migración corre dentro de un `BEGIN TRY / BEGIN TRAN`. Si cualquier stored procedure falla, se hace `ROLLBACK` de todo y se relanza el error, así la base nunca queda cargada a medias.
- **Control de la migración.** La vista `MEGA_DB.vw_Estado_Migracion` compara, para cada tabla principal, la cantidad esperada según la Maestra con la cantidad migrada. En la ejecución final todas las tablas dieron `OK`: no se descarta ninguna fila de la Maestra.

- **Índices y trigger al final.** Se crean después de la migración. Los índices quedan para el final porque así la carga es más rápida; el trigger, por el motivo que se explica en Inconsistencias – Alquileres superpuestos de la misma bicicleta.

Restricciones de integridad

- **CHECK de valores:** cantidades mayores a 0 y precios no negativos en los renglones; precio de venta y meses de garantía no negativos.
- **CHECK de fechas:**
  - En alquileres, la devolución (pactada o real) no puede ser anterior al inicio.
  - En reparaciones, la entrega (estimada o real) no puede ser anterior al ingreso.
- **UNIQUE:** sobre los identificadores de negocio de cada entidad.
- **UNIQUE de Modelo_Bicicleta:** se define sobre modelo + rodado + cuadro + color, sin incluir la marca. Se verificó que en la Maestra no hay modelos repetidos entre marcas distintas, por lo que esa combinación alcanza para identificar un modelo.
- **FKs:** todas las relaciones del DER.

Inconsistencias

Las siguientes inconsistencias vienen de la tabla Maestra. Se decidió **migrar los datos tal como están** y documentarlos en esta sección.

Productos repetidos en ventas y repuestos repetidos en reparaciones

- **Qué se encontró:** en algunos Detalles de Venta un mismo producto aparece en más de una Fila, no necesariamente con las mismas cantidades. En Reparacion_Repuesto pasa lo mismo.

```
select vd1.*
from MEGA_DB.Venta_Detalle vd1
join MEGA_DB.Venta_Detalle vd2
   on vd1.venta_id = vd2.venta_id
  and vd1.producto_id = vd2.producto_id
  and vd1.venta_detalle_id != vd2.venta_detalle_id
order by vd1.venta_id
```

	venta_detalle_id	venta_id	producto_id	cantidad	precio_unitario	subtotal
1	151864	9	10	5	1644998.43	8224992.17
2	151863	9	10	1	1644998.43	1644998.43
3	149903	9	7	4	144298.00	577192.00
4	149902	9	7	5	144298.00	721490.00
5	50233	28	3	4	105057.07	420228.29
6	50232	28	3	3	105057.07	315171.22

```
select *
from MEGA_DB.Reparacion_Repuesto rr1
join MEGA_DB.Reparacion_Repuesto rr2
   on rr1.reparacion_id = rr2.reparacion_id
  and rr1.repuesto_id = rr2.repuesto_id
  and rr1.reparacion_repuesto_id != rr2.reparacion_repuesto_id
```

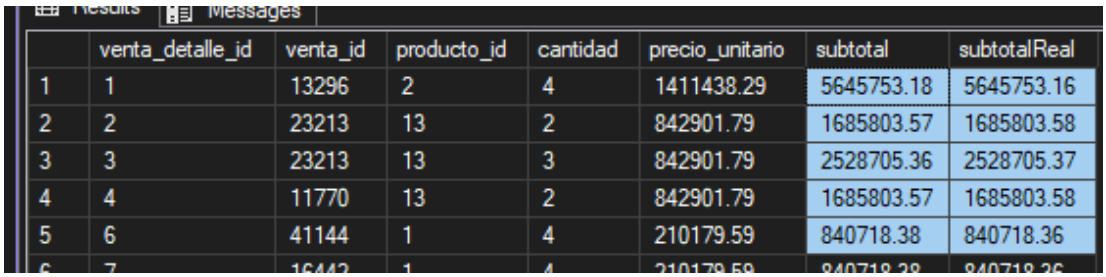
	reparacion_repuesto_id	reparacion_id	repuesto_id	cantidad	precio_unitario	subtotal
1	11742	5	3	1	6310.86	6310.86
2	11743	5	3	3	6310.86	18932.57
3	14186	25	10	1	43534.99	43534.99
4	14187	25	10	2	43534.99	87069.97
5	4908	27	2	3	27287.84	81863.52
6	4909	27	2	3	27287.84	81863.52

- **Decisión:** se migran todos los desgloses, así como están.
    - Los totales de las ventas y reparaciones coinciden con la suma de subtotales incluyendo a los repetidos. Por lo que se asume que son como distintos “ítems de venta” pero no necesariamente representan una inconsistencia.

Error en subtotales de venta

- **Qué se encontró:** en los Detalles de Venta hay registros donde si se calcula a mano el subtotal da distinto del valor precargado en la tabla Maestra

```
select *, precio_unitario * cantidad as subtotalReal
from MEGA_DB.Venta_Detalle
where subtotal != precio_unitario * cantidad
```

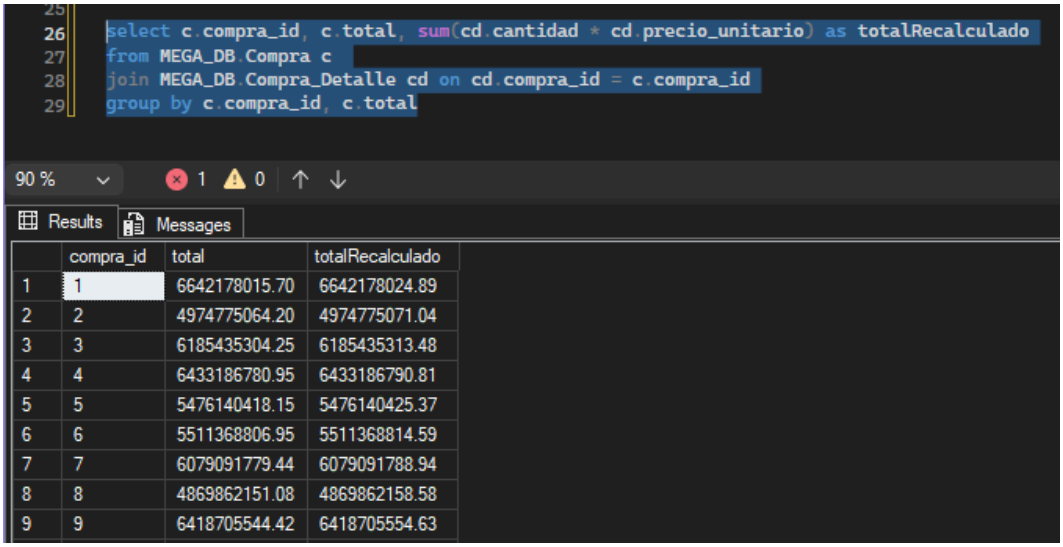


	venta_detalle_id	venta_id	producto_id	cantidad	precio_unitario	subtotal	subtotalReal
1	1	13296	2	4	1411438.29	5645753.18	5645753.16
2	2	23213	13	2	842901.79	1685803.57	1685803.58
3	3	23213	13	3	842901.79	2528705.36	2528705.37
4	4	11770	13	2	842901.79	1685803.57	1685803.58
5	6	41144	1	4	210179.59	840718.38	840718.36

- **Decisión:** ya que desconocemos el origen del error de cálculo y para evitar perder la información de la tabla original, se migran los importes sin recalcula.

Totales de compra

- **Qué se encontró:** todas las compras tienen un total distinto de la suma de cantidad * precio_unitario de sus detalles.



```
select c.compra_id, c.total, sum(cd.cantidad * cd.precio_unitario) as totalRecalculado
from MEGA_DB.Compra c
join MEGA_DB.Compra_Detalle cd on cd.compra_id = c.compra_id
group by c.compra_id, c.total
```

90 % 1 0

Results Messages

	compra_id	total	totalRecalculado
1	1	6642178015.70	6642178024.89
2	2	4974775064.20	4974775071.04
3	3	6185435304.25	6185435313.48
4	4	6433186780.95	6433186790.81
5	5	5476140418.15	5476140425.37
6	6	5511368806.95	5511368814.59
7	7	6079091779.44	6079091788.94
8	8	4869862151.08	4869862158.58
9	9	6418705544.42	6418705554.63

- **Decisión:** se migra el total de Maestra como viene. En los detalles no agregamos subtotal ya que no viene precargado nos parece suficiente tenerlo a nivel de Compra.

DNIs repetidos entre personas distintas

- **Qué se encontró:** hay 126 DNIs compartidos por personas con distinto nombre o apellido (la Query en pantalla devuelve 256 registros porque el repetido viene primero como “p1” y luego como “p2”).

```
select p1.dni as Dni1, p1.nombre as Nombre1, p1.apellido as Apellido1,
p2.dni as Dni2, p2.Nombre Nombre2, p2.apellido Apellido2
from MEGA_DB.Persona p1
join MEGA_DB.Persona p2
     on p1.dni = p2.dni
     and p1.persona_id != p2.persona_id
```

90 % 1 0

Results Messages

	Dni1	Nombre1	Apellido1	Dni2	Nombre2	Apellido2
1	22519131	Leonardo	Salazar	22519131	Juan Susana	Giménez
2	23257094	José Marcos	Valencia	23257094	Ana Ramiro	Silva
3	23358321	Carolina	Gómez	23358321	Ana Gabriela	Carrizo
4	23741429	María Camila	Martínez	23741429	Juan Yésica	Peralta
5	24133774	Juan Zulema	Castaño	24133774	Ana Abril	Santillán
6	24517818	José Julio	Giménez	24517818	Claudio	Gómez

- **Decisión:** el DNI no alcanza para identificar **unívocamente** a una persona, para eso está la PK de **persona**_id. Agregamos un Unique Constraint conformado por **dni + nombre + apellido + fecha_nacimiento**, así nunca se mezclan dos personas distintas.

Empleado – Local

- **Qué se encontró:** el enunciado dice que cada empleado pertenece a un único local, pero la Maestra no trae ese dato directamente.
  Tuvimos problemas con la relación 1 a 1 de Empleado/Local ya que los empleados vienen asociados cada uno a N locales. Estudiando los datos vimos que quitando el desglose por Alquileres nos queda uno a uno la relación *empleado-local*:

```
select distinct Empleado_Dni_Empleado, Empleado_Nombre_Empleado,
Local_Localidad, Local_Provincia
from gd_esquema.Maestra
where Empleado_Dni_Empleado is not null and Alquiler_Nro_Alquiler is null
order by Empleado_Dni_Empleado
```

Results Messages

Empleado_Dni_Empleado	Empleado_Nombre_Empleado	Local_Localidad	Local_Provincia
11839007	María Valeria	Pedro 1	Córdoba
13201886	Ana Jazmín	Carmen 4	Santa Fe
16683377	José Ramiro	Ciudad Victoria	Salta
25313992	José Laura	Carmen 4	Santa Fe
26756802	Juan Sebastián	Ciudad Victoria	Salta
27850531	Juan Olga	Carmen 4	Santa Fe
39542561	María Daniela	Pedro 1	Córdoba
39985567	Juan Alejandra	Pedro 1	Córdoba

Mientras que considerando los Alquileres enseguida se rompe el 1 a 1:

```
select distinct Empleado_Dni_Empleado, Empleado_Nombre_Empleado,
Local_Localidad, Local_Provincia
from gd_esquema.Maestra
where Empleado_Dni_Empleado is not null --and Alquiler_Nro_Alquiler is null
order by Empleado_Dni_Empleado
```

	Empleado_Dni_Empleado	Empleado_Nombre_Empleado	Local_Localidad	Local_Provincia
1	11839007	María Valeria	Pedro 1	Mendoza
2	11839007	María Valeria	Pedro 1	Córdoba
3	11839007	María Valeria	Carmen 4	Santa Fe
4	11839007	María Valeria	Ciudad Victoria	Salta
5	11839007	María Valeria	San Mercedes	Río Negro
6	13201886	Ana Jazmín	San Mercedes	Río Negro
7	13201886	Ana Jazmín	Pedro 1	Córdoba
8	13201886	Ana Jazmín	Ciudad Victoria	Salta
9	13201886	Ana Jazmín	Pedro 1	Mendoza
10	13201886	Ana Jazmín	Carmen 4	Santa Fe
11	16683377	José Ramiro	Ciudad Victoria	Salta

- **Decisión:** el local del empleado (Empleado.local_id, obligatorio) se toma de sus ventas, reparaciones y compras, sin contar los alquileres. Para todos los empleados da un solo local por cada uno.
  Queda la inconsistencia de empleados asociados a Alquileres a su vez asociados a Bicicletas de Alquiler que pertenecen a otros locales distintos al local del Empleado, pero no se modifica ningún dato original.

Alquileres superpuestos de la misma bicicleta

- **Qué se encontró:** en la tabla Maestra encontramos pares de registros donde distintos números de alquiler se asocian al mismo número de serie de bicicleta, pero el rango de tiempo de uno de los elementos del par se superpone con el rango del otro.
  Comprobamos esto asumiendo que un mismo numero de serie no debería tener un alquiler con fecha inicio mayor a la fecha de inicio y menor a la fecha real de devolución de otro alquiler de esta:

```
select b1.nro_serie,
a1.alquiler_id, a1.cliente_id, a1.fecha_inicio, a1.fecha_devolucion_real,
a2.alquiler_id, a2.cliente_id, a2.fecha_inicio, a2.fecha_devolucion_real
from MEGA_DB.Alquiler a1
join MEGA_DB.Alquiler a2
  on a1.bicicleta_alquiler_id = a2.bicicleta_alquiler_id
  and a1.alquiler_id != a2.alquiler_id
  and a1.nro_alquiler != a2.nro_alquiler
join MEGA_DB.Bicicleta_Alquiler b1
  on a1.bicicleta_alquiler_id = b1.bicicleta_alquiler_id
where a1.fecha_inicio < a2.fecha_inicio and a1.fecha_devolucion_real >
a2.fecha_inicio
```

```sql
select b1.nro_serie,
a1.alquiler_id, a1.cliente_id, a1.fecha_inicio, a1.fecha_devolucion_real,
a2.alquiler_id, a2.cliente_id, a2.fecha_inicio, a2.fecha_devolucion_real
from MEGA_DB.Alquiler a1
join MEGA_DB.Alquiler a2
    on a1.bicicleta_alquiler_id = a2.bicicleta_alquiler_id
    and a1.alquiler_id != a2.alquiler_id
    and a1.nro_alquiler != a2.nro_alquiler
join MEGA_DB.Bicicleta_Alquiler b1
    on a1.bicicleta_alquiler_id = b1.bicicleta_alquiler_id
where a1.fecha_inicio < a2.fecha_inicio and a1.fecha_devolucion_real > a2.fecha_inicio
```

Results | Messages

nro_serie	alquiler_id	cliente_id	fecha_inicio	fecha_devolucion_real	alquiler_id	cliente_id	fecha_inicio	fecha_devolucion_real
1H6EMQIJ3LO6	14	90276	2025-05-27 08:37:00.000000	2025-05-27 20:40:00.000000	48331	81826	2025-05-27 10:51:00.000000	2025-05-27 19:51:00.000000
1H6EMQIJ3LO6	14	90276	2025-05-27 08:37:00.000000	2025-05-27 20:40:00.000000	58732	102087	2025-05-27 10:08:00.000000	2025-05-27 19:02:00.000000
1H6EMQIJ3LO6	111	70490	2023-05-21 09:43:00.000000	2023-05-21 17:30:00.000000	40039	141818	2023-05-21 10:16:00.000000	2023-05-21 17:10:00.000000
1H6EMQIJ3LO6	155	16021	2024-09-13 08:28:00.000000	2024-09-13 19:31:00.000000	30360	37190	2024-09-13 09:57:00.000000	2024-09-13 19:53:00.000000
1H6EMQIJ3LO6	155	16021	2024-09-13 08:28:00.000000	2024-09-13 19:31:00.000000	32313	78109	2024-09-13 11:58:00.000000	2024-09-13 19:22:00.000000
1H6EMQIJ3LO6	221	120887	2025-04-09 08:51:00.000000	2025-04-09 19:43:00.000000	20114	61005	2025-04-09 11:16:00.000000	2025-04-09 17:24:00.000000
1H6EMQIJ3LO6	516	109365	2023-01-31 09:08:00.000000	2023-01-31 19:39:00.000000	1260	101992	2023-01-31 11:20:00.000000	2023-01-31 18:07:00.000000
1H6EMQIJ3LO6	516	109365	2023-01-31 09:08:00.000000	2023-01-31 19:39:00.000000	8308	33796	2023-01-31 09:29:00.000000	2023-01-31 19:34:00.000000

- **Decisión:** se migran todos los alquileres, porque descartarlos haría perder información. Para cumplir la regla sin afectar los registros existentes se creó el **trigger MEGA_DB.tr_Alquiler_Sin_Superposicion** (AFTER INSERT, UPDATE sobre Alquiler), que:
    - Rechaza un alquiler nuevo si su rango de fechas se superpone con otro de la misma bicicleta.
    - Si un alquiler no tiene fecha devolución real es porque no terminó. Al momento de comparar rangos de fechas de alquileres, que uno no haya terminado es como decir que su rango va desde su fecha de inicio hasta “infinito”, asique en el trigger pusimos:

      ```sql
      a.fecha_inicio < ISNULL(i.fecha_devolucion_real, '9999-12-31')
      ```

    - Se crea después de la migración porque, si existiera antes, la carga de los alquileres superpuestos de origen fallaría.